

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего
образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

**Лабораторная работа по
Вычислительной математике**

Интерполяция. Лагранж, Аиткен, Ньютона.

Вариант 56.

Студент,
группы 5130201/40003

_____ Четвергов И.С

Руководитель,
доцент к.т.н.

_____ Пак В. Г.

«_____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1	Инструменты и среда реализации	3
2	Постановка задачи	3
2.1	Полином Лагранжа	3
2.2	Схема Эйткена (Aitken)	4
2.3	Полином Ньютона и разделённые разности	5
2.4	Интерполяционные многочлены с постоянным шагом (Ньютон, Стирлинг) .	5
3	Результаты и их сравнение	7
4	Выводы	8

1 Инструменты и среда реализации

Вся реализация выполнена в среде Python и прототипирована в Jupyter Notebook. Для численных операций и матричных вычислений использовалась библиотека NumPy (быстрая и удобная для линейной алгебры), для интерполяции применялись функции из SciPy, а визуализация выполнялась с помощью matplotlib.

2 Постановка задачи

Тема: интерполяция

Цель: научиться вычислять приближённо табличную функцию с помощью интерполяционных многочленов

Задание:

1. Для данной табличной функции вычислить приближённое значение в точке интерполяции с помощью многочлена Лагранжа и схемы Эйткена.
2. Для табличной функции из задания 1 вычислить приближённые значения в точках интерполяции с помощью подходящего многочлена Ньютона с разделёнными разностями.
3. Для данной табличной функции вычислить приближённые значения в точках интерполяции с помощью подходящего многочлена Ньютона с конечными разностями (точки $x(1)$, $x(2)$) и наиболее подходящего из многочленов Гаусса, Стирлинга или Бесселя (точка x).

Входные данные

Ниже приведены исходные численные данные (табличная функция), используемые в работе:

$$\begin{aligned}x_0 &= -1.93, & y_0 &= 18.3456, \\x_1 &= -1.46, & y_1 &= 19.9472, \\x_2 &= -0.87, & y_2 &= 21.91492, \\x_3 &= -0.53, & y_3 &= 21.8492, \\x_4 &= -0.29, & y_4 &= 23.1236, \\x_5 &= -0.05, & y_5 &= 25.8484.\end{aligned}$$

Контрольная точка для сравнения методов задана как $x^* = -0.0$.

2.1 Полином Лагранжа

Полином Лагранжа может быть записан в нескольких эквивалентных формах. Одна из них через определитель для многочлена, построенного по всем узлам $x_0, \dots, x_n$:

$$L_n(x) = \frac{1}{x_n - x_0} \begin{vmatrix} L_n^{0, \dots, n-1}(x) & x_0 - x \\ L_n^{1, \dots, n}(x) & x_n - x \end{vmatrix}$$

Сокращенная форма записи

$$L_n(x) = \sum_{i=0}^n y_i l_{n,i}(x), \quad l_{n,i}(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Функция, реализующая эту формулу, называется `lagrange_interpolate` и принимает массивы `X`, `Y` и значение `point`. Ниже – полная реализация:

```

1 def lagrange_interpolate(X: np.ndarray,
2                           Y: np.ndarray,
3                           point: np.float64) -> np.float64:
4     n = X.shape[0]
5     if n != Y.shape[0]:
6         raise ValueError("X and Y must have the same length")
7
8     result = np.float64(0.0)
9     for i in range(n):
10         term = Y[i]
11         for j in range(n):
12             if i == j:
13                 continue
14             term *= (point - X[j]) / (X[i] - X[j])
15         result += term
16
17     return result

```

Листинг 1: Реализация Лагранжа

2.2 Схема Эйткена (Aitken)

Схема Эйткена строит таблицу приближений $P_{i,k}(x)$. Итерационная формула имеет вид

$$P_{i,0} = y_i, \quad P_{i,k}(x) = \frac{(x - x_{i+k})P_{i,k-1}(x) - (x - x_i)P_{i+1,k-1}(x)}{x_i - x_{i+k}}.$$

Эквивалентно это можно записать через детерминант размерности 2×2 :

$$P_{i,k}(x) = \frac{1}{x_i - x_{i+k}} \begin{vmatrix} P_{i,k-1}(x) & x_i - x \\ P_{i+1,k-1}(x) & x_{i+k} - x \end{vmatrix}.$$

Конечное значение $P_{0,n-1}(x)$ даёт интерполируемое значение в точке x . Ниже – полная реализация:

```

1 def aitken(X: np.ndarray,
2            Y: np.ndarray,
3            point: np.float64) -> np.float64:
4     n = X.shape[0]
5     if n != Y.shape[0]:
6         raise ValueError(f"X len: {n} != Y len:{Y.shape[0]}")
7
8     P = np.copy(Y.astype(np.float64))
9     for k in range(1, n):
10         for i in range(n - k):
11             term1 = (point - X[i + k]) * P[i]
12             term2 = (point - X[i]) * P[i + 1]
13             term3 = X[i] - X[i + k]
14             P[i] = (term1 - term2) / term3
15
16     return P[0]

```

Листинг 2: Реализация схемы Эйткена

2.3 Полином Ньютона и разделённые разности

Разделённые разности строят таблицу

$$f[x_i] = y_i, \quad f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Полином Ньютона имеет вид

$$P_N(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

Реализация строит двумерную таблицу разделённых разностей и затем вычисляет значение в точке. Ниже – полная реализация:

Входные данные: Контрольные точки: $x^{(1)} = -0.18$, $x^{(2)} = -1.50$.

```
1 def newton_interpolate(X: np.ndarray,
2                       Y: np.ndarray,
3                       point: np.float64) -> np.float64:
4     n = X.shape[0]
5     if n != Y.shape[0]:
6         raise ValueError("X and Y must have the same length")
7
8     fdd = np.zeros((n, n), dtype=np.float64)
9     for i in range(n):
10         fdd[i][0] = Y[i]
11
12     for j in range(1, n):
13         for i in range(n - j):
14             term1 = fdd[i + 1][j - 1]
15             term2 = fdd[i][j - 1]
16             term3 = X[i + j] - X[i]
17             fdd[i][j] = (term1 - term2) / term3
18
19     result = fdd[0][0]
20     for i in range(1, n):
21         term = fdd[0][i]
22         for j in range(i):
23             term *= (point - X[j])
24         result += term
25
26     return result
```

Листинг 3: Реализация Ньютона

2.4 Интерполяционные многочлены с постоянным шагом (Ньютон, Стирлинг)

Для выполнения третьей части задания мы аппроксимируем сетку узлов как квази-равномерную с шагом $h \approx \max(\Delta x_i)$. В таком случае для интерполирования применимы формулы Ньютона через конечные разности, а также многочлены типа Гаусса, Стирлинга или Бесселя.

Многочлен Стирлинга (основанный на центральных конечных разностях) применяется, когда точка x близка к центральным узлам. Для упрощения используется средний шаг h :

$$u = \frac{x - x_{\text{mid}}}{h}$$

Формула с использованием центральных разностей имеет вид:

$$P(x) = y_{\text{mid}} + u \frac{\Delta y_{\text{mid}} + \Delta y_{\text{mid}-1}}{2} + \frac{u^2}{2} \Delta^2 y_{\text{mid}-1} + \frac{u(u^2 - 1)}{3!} \frac{\Delta^3 y_{\text{mid}-1} + \Delta^3 y_{\text{mid}-2}}{2} + \dots$$

Ниже приведена реализация функции вычисления многочлена Стирлинга:

```

1 def stirling_interpolate(X: np.ndarray,
2                           Y: np.ndarray,
3                           point: np.float64) -> np.float64:
4     n = len(X)
5     mid = n // 2 if n % 2 != 0 else n // 2 - 1
6
7     h = np.mean(np.diff(X))
8     u = (point - X[mid]) / h
9
10    dy = np.zeros((n, n), dtype=np.float64)
11    dy[:, 0] = Y
12    for j in range(1, n):
13        for i in range(n - j):
14            dy[i, j] = dy[i + 1, j - 1] - dy[i, j - 1]
15
16    res = dy[mid, 0]
17    if mid + 1 < n and mid >= 0:
18        res += (u / 2) * (dy[mid, 1] + dy[mid-1, 1])
19    if mid - 1 >= 0:
20        res += (u**2 / 2) * dy[mid-1, 2]
21    if mid - 1 >= 0 and mid + 2 < n:
22        res += (u * (u**2 - 1) / 12) * (dy[mid-1, 3] + dy[mid-2, 3])
23
24    return res

```

Листинг 4: Реализация многочлена Стирлинга

Многочлены Ньютона с конечными разностями делятся на первую формулу (для точек вблизи начала таблицы) и вторую формулу (для точек вблизи конца таблицы). Для вычисления разделённых и конечных разностей заданы две дополнительные точки: $x^{(2)} = -1.50$ (ближе к началу, используем первую формулу «вперёд») и $x^{(1)} = -0.18$ (ближе к концу, используем вторую формулу «назад»).

Ниже приведена реализация функции вычисления многочленов Ньютона через конечные разности:

```

1 def newton_finite_diff(X: np.ndarray,
2                         Y: np.ndarray,
3                         target: np.float64,
4                         direction: str = 'forward') -> np.float64:
5     n = len(X)
6     h = np.mean(np.diff(X))
7     dy = np.zeros((n, n), dtype=np.float64)
8     dy[:, 0] = Y
9     for j in range(1, n):
10        for i in range(n - j):
11            dy[i, j] = dy[i + 1, j - 1] - dy[i, j - 1]
12
13    if direction == 'forward':
14        u = (target - X[0]) / h
15        res = dy[0, 0]

```

```

16     term = 1.0
17     for i in range(1, n):
18         term *= (u - i + 1) / i
19         res += term * dy[0, i]
20     else:
21         u = (target - X[-1]) / h
22         res = dy[n - 1, 0]
23         term = 1.0
24         for i in range(1, n):
25             term *= (u + i - 1) / i
26             res += term * dy[n - 1 - i, i]
27     return res

```

Листинг 5: Реализация Ньютона (конечные разности)

3 Результаты и их сравнение

Вычисленные значения интерполяционного многочлена в контрольной точке $x^* = -0.0$:

Лагранж: 26.519558662617705,
 Эйткен: 26.519558662617698,
 Ньютон (разделённые разности): 26.519558662617687,
 Стирлинг ($x = -0.0$): 26.376823376288675.

Стирлинг дает небольшое отклонение, так как сетка узлов неравномерная, а формула вычислялась с использованием усреднённого шага h .

Дополнительные вычисленные значения в других точках по многочленам Ньютона (конечные разности):

Ньютон, 2-я формула ($x^{(1)} = -0.18$): 24.23742468,
 Ньютон, 1-я формула ($x^{(2)} = -1.50$): 19.82481498.

Разница между основными методами (Лагранж, Эйткен, Ньютон с разделёнными разностями) не превышает $2e-14$; машинный эпсилон для двойной точности (double) равен примерно $2.22e-16$.

Эталон и дополнительные значения:

scipy lagrange: 26.519558662617705,
 my lagrange: 26.519558662617705,
 difference: 0.0.

Вывод: все три реализованных метода дают согласующиеся значения интерполяционного многочлена в пределах численной погрешности вычислений.

Математически Лагранж, схема Эйткена и многочлен Ньютона задают один и тот же интерполяционный многочлен степени n , следовательно их графики совпадают; на общем масштабе все кривые накладываются и визуально образуют "монолит" (единый слой), что объясняет отсутствие видимых отличий на сводном графике.

На Рисунке 1 показано сравнение графиков интерполяции для всех методов (включая эталон от SciPy):

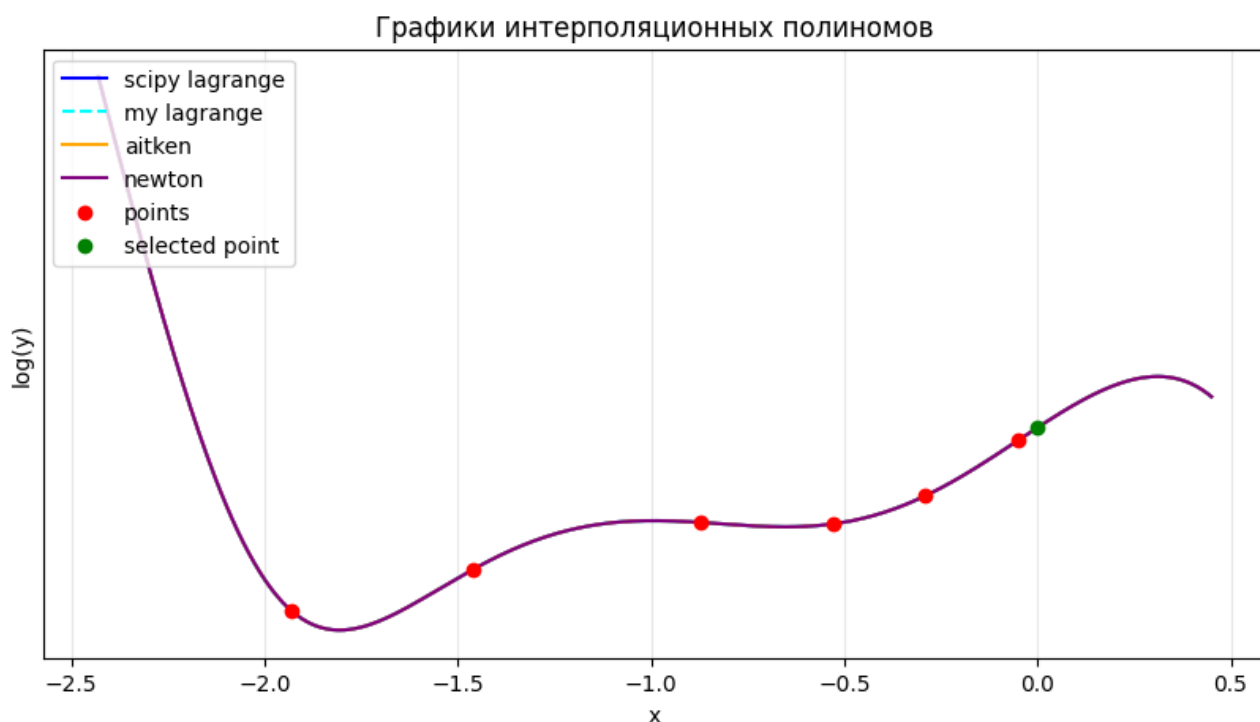


Рис. 1: Сравнение интерполяционных методов (Лагранж, Эйткен, Ньютон) и эталон SciPy; кривые накладываются и видны как единый слой.

4 Выводы

- Реализованные методы интерполяции (Лагранж, Эйткен, Ньютон через разделённые разности) эквивалентны по математической конструкции и при корректной реализации для произвольной сетки узлов дают одинаковые значения интерполяционного полинома с точностью до машинной погрешности.
- Использование формул с конечными разностями (Стирлинг, Ньютон вперёд/назад) на существенно неравномерной сетке при допущении об "усредненном" шаге приводит к появлению методической погрешности, однако позволяет получить приближённые оценки, если неравномерность сетки мала.
- Применение многочлена Стирлинга к точке $x = -0.0$ носит оценочный характер, так как точка находится вблизи правого края таблицы, а не в ее середине, для которой оптимизированы центральные разностные схемы. Также на погрешность повлияло ограничение по учету высоких порядков конечных разностей.